

Relatório de Auditoria de Segurança — Foco Semanal

Data: 29/08/2026

Escopo: repositório completo (código + SQL Supabase + rotas API)

Stack detectada

TypeScript · Next.js 16 (App Router) · React 19 · Supabase Auth + Postgres (cliente @supabase/supabase-js / SSR) · Tailwind 4 · deploy documentado em Vercel (sem Docker/CI/Helm/Terraform no repo).

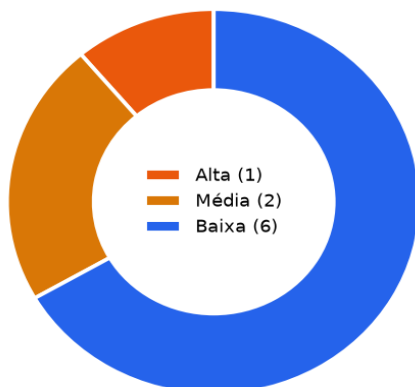
Nota metodológica (mapeamento das categorias)

1) **Banco sem tranca** → RLS Supabase + filtro user_id no sync. 2) **Permissão no navegador** → gates isAdmin/role no React cruzados com RPC/API. 3) **IDOR** → inventário de handlers (api/access/invite, auth/callback) + mutações PostgREST. 4) **Chaves expostas** → código, .env.example, docs, histórico git. 5) **XSS** → dangerouslySetInnerHTML, note-html, renderização de lembretes. Somente achados verificados no código; ausências também registradas.

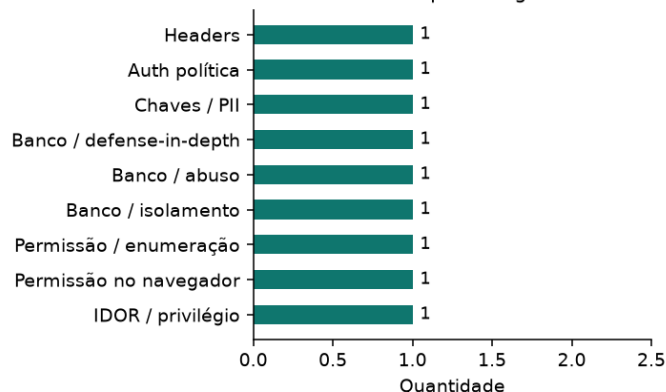
1. Resumo executivo

Total de achados acionáveis: **9** — crítica 0, alta 1, média 2, baixa 6, informativa 0. Nenhum vazamento cross-tenant sob RLS foi confirmado. Principal risco: takeover via sobrescrita de senha no convite.

Achados por severidade



Achados por categoria



2. Pontos fortes

- ✓ **RLS em todas as tabelas de dados**

supabase/schema.sql:178–200 — `auth.uid() = user_id + current_user_has_access()`

- ✓ **Allowlist admin só para owners**

supabase/schema.sql:201–202 — `current_user_is_access_admin()`

- ✓ **RPCs de acesso com security definer e search_path vazio**

supabase/schema.sql:123–149

- ✓ **Rota de convite revalida admin no servidor**

src/app/api/access/invite/route.ts:37–48

- ✓ **Service role isolada (server-only)**

src/lib/supabase/admin.ts:1 + src/lib/env.ts

- ✓ **Sem segredos hardcoded / .env no git**

.env.example placeholders; .gitignore .env*; histórico sem .env commitado

- ✓ **Sem XSS verificado**

dangerouslySetInnerHTML só com THEME_INIT_SCRIPT estático; notas via texto / plainTextFromHtml

- ✓ **Open redirect mitigado**

src/lib/safe-path.ts usado em auth/callback e proxy

- ✓ **Sync nuvem sob RLS**

src/lib/supabase/sync.ts — deletes/inserts com user_id + policies auth.uid()

Pontos fracos (riscos centrais)

- Owner pode redefinir senha de qualquer Auth user via convite (alta).
- Remoção/alteração de owners da allowlist não está travada no banco (média).
- Enumeração da allowlist por anon + UX de login (média).

3. Achados detalhados por categoria

Sev.	Arquivo:linha	Descrição
ALTA	src/app/api/access/invite/route.ts:89–129	F-01 — Convite sobrescreve senha de qualquer conta Auth existente Após falha de createUser por e-mail já registrado, a rota usa admin.updateUserById e define a senha enviada pelo owner. Qualquer owner autenticado pode assumir a conta de outro usuário (inclusive outro owner) sem prova de posse do e-mail.
MÉDIA	src/components/AccessManagement.tsx:103–112	F-02 — Proteção de owner na revogação existe só na UI revoke() retorna cedo se item.role === 'owner', mas a política RLS access_allowlist_admin permite DELETE/UPDATE a qualquer owner. Não há regra no banco impedindo apagar/alterar outro owner.
MÉDIA	supabase/schema.sql:123–130, 168	F-03 — RPC is_email_allowed exposta a anon permite enumerar allowlist grant execute ... to anon + checkEmailAccess no login distingue 'sem acesso' de 'senha errada', permitindo descobrir quais e-mails estão na allowlist.
BAIXA	supabase/schema.sql:92–98	F-04 — FK sticky_notes.column_id sem garantir mesmo user_id column_id referencia note_columns(id) sem check de que note_columns.user_id = sticky_notes.user_id. RLS impede leitura cruzada, mas permite referência órfã/cruzada na escrita.
BAIXA	supabase/schema.sql:151–169	F-05 — request_demo_access gravável por anon sem rate limit SECURITY DEFINER insert em access_requests concedida a anon. UI atual usa WhatsApp, mas a RPC permanece aberta a spam.
BAIXA	supabase/schema.sql:205–208	F-06 — handle_new_user com search_path = public Outras funções de acesso usam search_path = ". O trigger de signup ainda usa public — padrão clássico de hijack se um atacante puder criar objetos em public (improvável no Supabase padrão).
BAIXA	supabase/schema.sql:173–175	F-07 — E-mail do owner hardcoded no schema/migrations insert ... values ('davicoruda@gmail.com', 'owner'). Não é segredo criptográfico, mas expõe PII se o repositório for público.
BAIXA	src/app/api/access/invite/route.ts:12–14	F-08 — Política de senha mínima fraca (6 caracteres) validPassword exige apenas length >= 6; espelhado no login.
BAIXA	next.config.ts:3–5	F-09 — Sem headers de segurança na configuração Next nextConfig vazio: sem CSP, X-Frame-Options, Referrer-Policy etc. no app (podem existir no provedor de hosting).

Detalhamento arquivo a arquivo

F-01 [ALTA] · IDOR / privilégio

Convite sobrescreve senha de qualquer conta Auth existente

Local: src/app/api/access/invite/route.ts:89–129

Por que é explorável: Owner chama POST /api/access/invite com e-mail de vítima já em auth.users e senha escolhida → takeover imediato no projeto.

Impacto: Conta takeover dentro do projeto; abuso entre co-owners.

F-02 [MÉDIA] · Permissão no navegador

Proteção de owner na revogação existe só na UI

Local: src/components/AccessManagement.tsx:103–112

Por que é explorável: Owner malicioso chama PostgREST delete/update em access_allowlist para remover outro owner ou alterar roles.

Impacto: Escalação/remoção de privilégios entre co-owners.

F-03 [MÉDIA] · Permissão / enumeração

RPC is_email_allowed exposta a anon permite enumerar allowlist

Local: supabase/schema.sql:123–130, 168

Por que é explorável: Com a anon key pública, chamar rpc is_email_allowed em massa ou observar mensagens do LoginScreen.

Impacto: Vazamento de quem tem acesso ao sistema (PII / targeting).

F-04 [BAIXA] · Banco / isolamento

FK sticky_notes.column_id sem garantir mesmo user_id

Local: supabase/schema.sql:92–98

Por que é explorável: Usuário allowlisted insere sticky com column_id de outro usuário (se souber o UUID). Não lê dados alheios; risco de integridade.

Impacto: Integridade referencial entre tenants; não é IDOR de leitura.

F-05 [BAIXA] · Banco / abuso

request_demo_access gravável por anon sem rate limit

Local: supabase/schema.sql:151–169

Por que é explorável: Chamadas anônimas repetidas enchem access_requests.

Impacto: Spam operacional para owners; DoS leve na tabela.

F-06 [BAIXA] · Banco / defense-in-depth

handle_new_user com search_path = public

Local: supabase/schema.sql:205–208

Por que é explorável: Condicional a privilégio de criar objetos em public.

Impacto: Baixo no setup típico Supabase; defesa em profundidade.

F-07 [BAIXA] · Chaves / PII

E-mail do owner hardcoded no schema/migrations

Local: supabase/schema.sql:173–175

Por que é explorável: Leitura do repo → targeting.

Impacto: Privacidade / superfície de engenharia social.

F-08 [BAIXA] · Auth política

Política de senha mínima fraca (6 caracteres)

Local: src/app/api/access/invite/route.ts:12–14

Por que é explorável: Senhas curtas/previsíveis em convites e contas.

Impacto: Facilita credential stuffing / adivinhação.

F-09 [BAIXA] · Headers**Sem headers de segurança na configuração Next**

Local: next.config.ts:3–5

Por que é explorável: Depende de config da edge; baseline do app não reforça.

Impacto: Clickjacking / XSS mitigation reduzida se hosting não cobrir.

Categorias sem achado explorável

- **XSS:** nenhum sink com input de usuário; `noteHtmlFromStored` não está ligado a `dangerouslySetInnerHTML`; lembretes usam texto puro.
- **Chaves hardcoded:** nenhum segredo de produção no código; `service_role` só no servidor; defaults `${VAR:-secret}` ausentes.

4. Recomendações priorizadas

P1 — Corrigir sobrescrita de senha em /api/access/invite (F-01).

P2 — Travar DELETE/UPDATE de owners na allowlist no Postgres (F-02).

P3 — Eliminar enumeração de e-mails (mensagem genérica + restringir RPC) (F-03).

P4 — Endurecimento SQL: FK sticky, RPC demo, search_path do trigger (F-04–F-06).

P5 — Remover e-mail pessoal do seed; reforçar senha e headers (F-07–F-09).

5. Issues para o GitHub

Copie cada bloco abaixo para criar uma issue. Achados relacionados foram agrupados.

--- ISSUE 1 ---

```
## [Segurança] Convite redefine senha de contas Auth existentes

**Labels sugeridas:** `security, alta`

### Problema
- **F-01 (alta):** Após falha de createUser por e-mail já registrado, a rota usa admin.updateUserById e define a senha enviada pelo owner. Qualquer owner autenticado pode assumir a conta de outro usuário (inclusive outro owner) sem prova de posse do e-mail.

### Por que é explorável
- Owner chama POST /api/access/invite com e-mail de vítima já em auth.users e senha escolhida → takeover imediato no projeto.

### Evidência
- `src/app/api/access/invite/route.ts:89-129` – Convite sobrescreve senha de qualquer conta Auth existente

### Impacto
- Conta takeover dentro do projeto; abuso entre co-owners.

### Sugestão de correção
- Não atualizar senha de contas existentes; ou exigir fluxo de reset por e-mail; bloquear update se o usuário já confirmou login; nunca sobrescrever senha de role=owner.

### Critérios de aceite
- [ ] Reconvidar e-mail existente NÃO altera a senha Auth
- [ ] Ou só cria usuário novo; existente recebe link de reset
- [ ] Teste: owner A não redefine senha de owner B via /api/access/invite
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
## [Segurança] Owner rows da allowlist protegidos só na UI

**Labels sugeridas:** `security, média`

### Problema
- **F-02 (média):** revoke() retorna cedo se item.role === 'owner', mas a política RLS access_allowlist_admin permite DELETE/UPDATE a qualquer owner. Não há regra no banco impedindo apagar/alterar outro owner.

### Por que é explorável
- Owner malicioso chama PostgREST delete/update em access_allowlist para remover outro owner ou alterar roles.

### Evidência
- `src/components/AccessManagement.tsx:103-112` – Proteção de owner na revogação existe só na UI

### Impacto
- Escalação/remoção de privilégios entre co-owners.

### Sugestão de correção
- Policy/trigger: impedir DELETE/UPDATE de linhas com role='owner' por outro usuário (exceto self-leave consciente, se desejado).

### Critérios de aceite
- [ ] DELETE de e-mail com role=owner via API falha para outros owners
- [ ] UI continua escondendo o botão, mas o backend é a fonte da verdade
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
## [Segurança] Enumeração de e-mails da allowlist via RPC anon

**Labels sugeridas:** `security, média`
```

```
### Problema
- **F-03 (média):** grant execute ... to anon + checkEmailAccess no login distingue 'sem acesso' de 'senha errada', permitindo descobrir quais e-mails estão na allowlist.

### Por que é explorável
- Com a anon key pública, chamar rpc is_email_allowed em massa ou observar mensagens do LoginScreen.

### Evidência
- `supabase/schema.sql:123-130, 168` – RPC is_email_allowed exposta a anon permite enumerar allowlist

### Impacto
- Vazamento de quem tem acesso ao sistema (PII / targeting).

### Sugestão de correção
- Mensagem genérica no login; rate-limit; remover grant anon ou exigir prova (CAPTCHA / challenge) antes de revelar allowlist.

### Critérios de aceite
- [ ] Login não revela se o e-mail está na allowlist
- [ ] Chamadas anônimas à RPC estão rate-limited ou revogadas
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
## [Segurança] Endurecimento SQL: FK sticky, RPC demo, search_path
```

```
**Labels sugeridas:** `security, baixa`
```

```
### Problema
- **F-04 (baixa):** column_id referencia note_columns(id) sem check de que note_columns.user_id = sticky_notes.user_id. RLS impede leitura cruzada, mas permite referência órfã/cruzada na escrita.
- **F-05 (baixa):** SECURITY DEFINER insert em access_requests concedida a anon. UI atual usa WhatsApp, mas a RPC permanece aberta a spam.
- **F-06 (baixa):** Outras funções de acesso usam search_path = ''. O trigger de signup ainda usa public – padrão clássico de hijack se um atacante puder criar objetos em public (improvável no Supabase padrão).
```

```
### Por que é explorável
- Usuário allowlisted insere sticky com column_id de outro usuário (se souber o UUID). Não lê dados alheios; risco de integridade.
- Chamadas anônimas repetidas enchem access_requests.
- Condicional a privilégio de criar objetos em public.
```

```
### Evidência
- `supabase/schema.sql:92-98` – FK sticky_notes.column_id sem garantir mesmo user_id
- `supabase/schema.sql:151-169` – request_demo_access gravável por anon sem rate limit
- `supabase/schema.sql:205-208` – handle_new_user com search_path = public
```

```
### Impacto
- Integridade referencial entre tenants; não é IDOR de leitura.
- Spam operacional para owners; DoS leve na tabela.
- Baixo no setup típico Supabase; defesa em profundidade.
```

```
### Sugestão de correção
- Trigger/check ou composite FK garantindo mesmo user_id.
- Revogar grant anon se não usado; ou rate-limit / CAPTCHA.
- set search_path = '' e qualificar public.profiles etc.
```

```
### Critérios de aceite
- [ ] INSERT sticky com column_id de outro user_id é rejeitado
- [ ] Anon não consegue floodar access_requests sem limite
- [ ] Função handle_new_user declara search_path vazio ou fixo seguro
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```
## [Segurança] Remover e-mail pessoal do seed versionado
```

```
**Labels sugeridas:** `security, baixa`
```

```
### Problema
- **F-07 (baixa):** insert ... values ('davicoruda@gmail.com', 'owner'). Não é segredo criptográfico, mas
```

expõe PII se o repositório for público.

Por que é explorável
- Leitura do repo → targeting.

Evidência
- `supabase/schema.sql:173-175` – E-mail do owner hardcoded no schema/migrations

Impacto
- Privacidade / superfície de engenharia social.

Sugestão de correção
- Seed via variável de ambiente / SQL manual pós-deploy.

Critérios de aceite
- [] Schema versionado não contém e-mail pessoal real

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Política de senha e headers HTTP de baseline

Labels sugeridas: `security, baixa`

Problema
- **F-08 (baixa):** validPassword exige apenas length >= 6; espelhado no login.
- **F-09 (baixa):** nextConfig vazio: sem CSP, X-Frame-Options, Referrer-Policy etc. no app (podem existir no provedor de hosting).

Por que é explorável
- Senhas curtas/previsíveis em convites e contas.
- Depende de config da edge; baseline do app não reforça.

Evidência
- `src/app/api/access/invite/route.ts:12-14` – Política de senha mínima fraca (6 caracteres)
- `next.config.ts:3-5` – Sem headers de segurança na configuração Next

Impacto
- Facilita credential stuffing / adivinhação.
- Clickjacking / XSS mitigation reduzida se hosting não cobrir.

Sugestão de correção
- Mínimo 10-12; opcional complexidade; alinhar Auth Supabase.
- Definir headers em next.config ou no painel Vercel.

Critérios de aceite
- [] API e UI rejeitam senhas abaixo do novo mínimo
- [] Resposta HTTP inclui ao menos frame-ancestors/CSP básica

--- FIM ISSUE 6 ---